

--{ python для пентестера }--

Ethical Hacking Tutorial Group The Codeby

представляет



PYTHON

«Для Пентестера»

Programming Language..



-- { ПРОДВИНУТЫЙ УРОВЕНЬ } --

Модуль requests

Requests – это модуль Python, который используется для отправки всех видов HTTP-запросов. [Документация по библиотеке.](#)

Модуль не является встроенным, поэтому его нужно сначала установить командой в терминале `pip install requests`.

HTTP методы GET и POST самые часто используемые.

Метод GET в HTTP используется для получения информации от сервера по заданному URI.

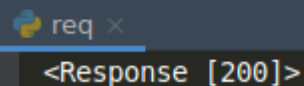
Метод POST используется для отправки данных на сервер, например, из HTML форм, которые заполняет посетитель сайта.

Создание запроса GET

Метод `requests.get()` возвращает объект **Response**.

```
import requests

res = requests.get('https://codeby.net/')
print(res)
```



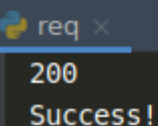
```
req x
<Response [200]>
```

Проверить успешность запроса можно с помощью `status_code`.

```
import requests

res = requests.get('https://codeby.net/')

if res.status_code == 200:
    print(res.status_code)
    print('Success!')
elif res.status_code == 404:
    print('Not Found!')
    print(res.status_code)
```



```
req x
200
Success!
```

Коды состояния:

1XX – информация

2XX – успешно

3XX – перенаправление

4XX — Ошибка клиента

5XX — Ошибка сервера

Если мы наберём адрес несуществующей страницы, то получим ошибку 404.

```
import requests

res = requests.get('https://codeby.net/blabla')
print(res)
if res.status_code == 200:
    print('Success!')
    print(res.status_code)
elif res.status_code == 404:
    print('Not Found!')
    print(res.status_code)
```

req ×

<Response [404]>
Not Found!
404

Мы также можем перехватить ошибку бросив исключение.

```
import requests

res = requests.get('https://codeby.net/s')
print(res)
try:
    res.raise_for_status()
except Exception as e:
    print('Ошибка при загрузке страницы: ', str(e), sep='\n')
```

req_2 ×

<Response [404]>
Ошибка при загрузке страницы:
404 Client Error: Not Found for url: <https://codeby.net/s>

Получить весь HTML-код можно с помощью `res.text`.

```
import requests

res = requests.get('https://codeby.net/')
print(res.text)
```

req_2 x

```
<!DOCTYPE html>
<html id="XF" lang="ru-RU" dir="LTR" style="font-size: 62.5%;
<head>
<meta charset="utf-8" />
<meta http-equiv="X-UA-Compatible" content="IE=Edge" />
<meta name="viewport" content="width=device-width, initial-sca
<title>Форум информационной безопасности - Codeby.net</title>
```

Если какой-либо ресурс отдаст ответ в виде крякозябр, значит кодировка не смогла корректно определиться. В таком случае её можно указать принудительно.

```
import requests

res = requests.get('https://codeby.net/')
res.encoding = 'utf-8'
print(res.text)
```

resp x

```
<!DOCTYPE html>
<html id="XF" lang="ru-RU" dir="LTR" style="font-size: 62.5%;
<head>
```

Узнать какую кодировку использует ресурс можно с помощью `encoding`.

```
import requests

res = requests.get('https://codeby.net/')
print(res.encoding)
```

resp x

```
utf-8
```

Если нас интересуют HTTP-заголовки, то получить их можно спомощью `res.headers`. Заголовки выводятся в виде словаря.

```
import requests

res = requests.get('https://codeby.net/')
print(res.headers)
```

req_2 x

```
{'Date': 'Tue, 09 Oct 2019 08:54:02 GMT', 'Content-Type': 'text/html; cha
```

А поскольку это словарь, то мы можем получить и только определённый заголовок, указав его в квадратных скобках. Стоит отметить, что имена заголовков HTTP нечувствительны к регистру.

```
import requests

res = requests.get('https://codeby.net/')
print(res.headers['server'])
```

resp ×
cloudflare

Чтобы добавить свои HTTP-заголовки в запрос, мы можем передать их в dict для параметра headers. Аналогично, мы также можем отправлять свои собственные файлы cookie на сервер, используя dict, переданный в параметр cookie.

Все запросы можно отработать на тестовом ресурсе.

```
import requests

url = 'https://httpbin.org/cookies'
cookies = {'secret': 'blablabla'}
req = requests.get(url, cookies=cookies)
print(req.text)
```

4 ×

```
{
  "cookies": {
    "secret": "blablabla"
  }
}
```

Или сразу несколько заголовков.


```
import requests

url = 'https://httpbin.org/anything'
headers = {'user-agent': 'super-agent-007'}
cookies = {'secret': 'blablabla'}
req = requests.get(url, headers=headers, cookies=cookies)
print(req.text)
```

reg_7 ×

```
{
  "args": {},
  "data": "",
  "files": {},
  "form": {},
  "headers": {
    "Accept": "*/*",
    "Accept-Encoding": "gzip, deflate",
    "Cookie": "secret=blablabla",
    "Host": "httpbin.org",
    "User-Agent": "super-agent-007",
    "X-Amzn-Trace-Id": "Root=1-5edff4ec-5f1e659a5437215e6e931ef6"
```

Не всегда полученная страница является именно той, адрес которой вы запросили. Перенаправления (редиректы) являются обычным делом.

```
import requests

url = 'http://yandex.ru'
res = requests.get(url)
print(res.status_code)
print(res.history)
```

reg_8 ×

```
200
[<Response [302]>]
```

Параметр `allow_redirects=False` отключает обработку перенаправления.

```
import requests

url = 'http://yandex.ru'
res = requests.get(url, allow_redirects=False)
print(res.status_code)
print(res.history)
```

reg_8 ×

```
302
[]
```

User-Agent

Нужно сказать несколько слов по поводу этого заголовка: его задача состоит в том, чтобы отправить серверу информацию о системе, браузере и версии ПО, используемого для выполнения запроса.

Общий вид выглядит следующим образом:

```
User-Agent: <product> / <product-version> <comment>
```

Например, Chrome в Windows 10 заполняет его так:

```
user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/117.0.0.0
Safari/537.36
```

Mozilla/5.0 – пункт, подтверждающий совместимость с Mozilla – дань традиции
(Windows NT 10.0; Win64; x64) – информация через точку с запятой о версии операционной системы и её разрядности

Дальше идёт информация о библиотеке/фреймворке и версии браузера.

Важно понимать, что на основе этой информации, вкупе с IP адресом и заголовками cookie сервер «узнаёт» вас. С одной стороны – это огромный плюс, ведь мы получаем одну версию страницы для телефона, другую – для планшета, третью – для браузера в ноутбуке, но с другой стороны – эти данные легко подменить, выдав себя за нового или вообще за другого пользователя.

Изменить заголовок User-Agent можно вручную, заполнив его строкой с нужной нам версией браузера, благо, эти версии стандартны. Можно найти их на [github](#) или по [ссылке](#)

Есть и более простой способ – использовать разнообразные модули, уже написанные для этих целей сообществом python среди сотен тысяч проектов на ресурсе pypi.org. Например, [fake_user_agent](#) Устанавливается он, как и любой другой модуль через pip либо через поиск в PyCharm.

В использовании нет ничего сложного – импортируем объект user_agent из модуля fake_user_agent, вызываем в коде, передав в аргументе название браузера

```
1 from fake_user_agent import user_agent
2
3 new_useragent = user_agent('chrome')
4 print(new_useragent)
5
```

На выходе получаем строку

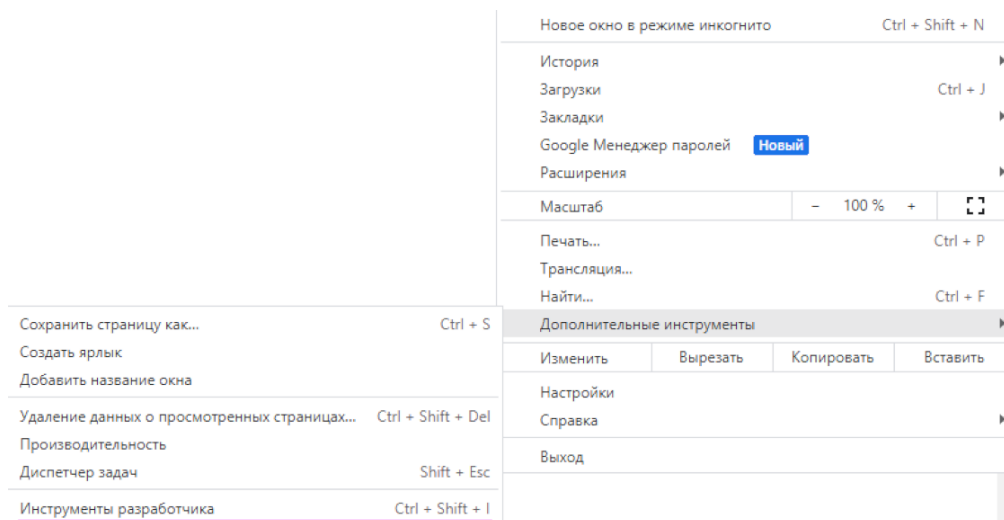
```
Mozilla/5.0 (Windows NT 6.2; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/32.0.1667.0 Safari/537.36
```

Её и добавляем в параметры запроса. Изменяя этот заголовок при каждой отправке данных к серверу, мы можем каждый раз притворяться новым пользователем.

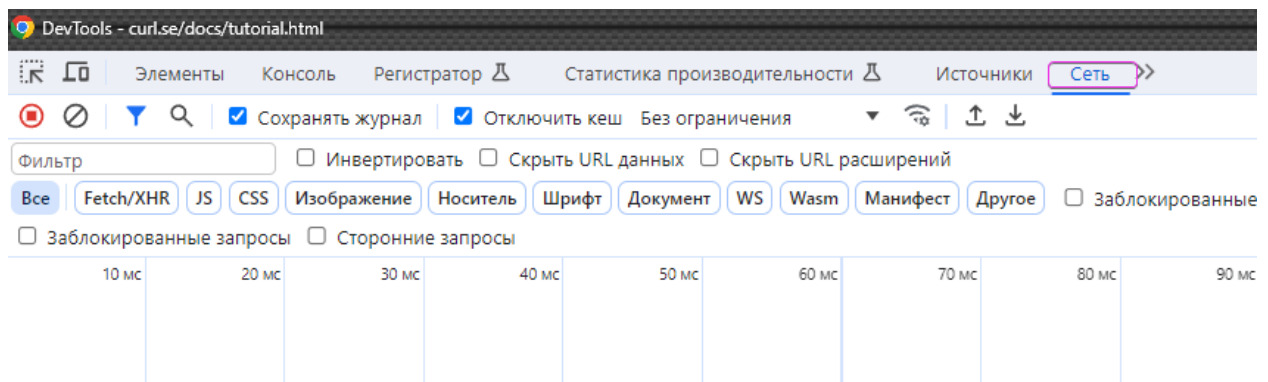
Повторение запросов браузера

Но случаются ситуации, когда необходимо получить ответ сервера аналогичный запросу из браузера. Зачастую он содержит множество заголовков со своими параметрами и вручную вводить их каждый раз довольно утомительно.

Предположим, после авторизации в браузере мы получили различные заголовки в том числе cookie и хотим использовать их в нашей программе на python. Тут нам пригодится технология [curl](#) – она позволит сформировать строку запроса, содержащую **все заголовки**. Чтобы получить её, открываем браузер и переходим в инструменты разработчика



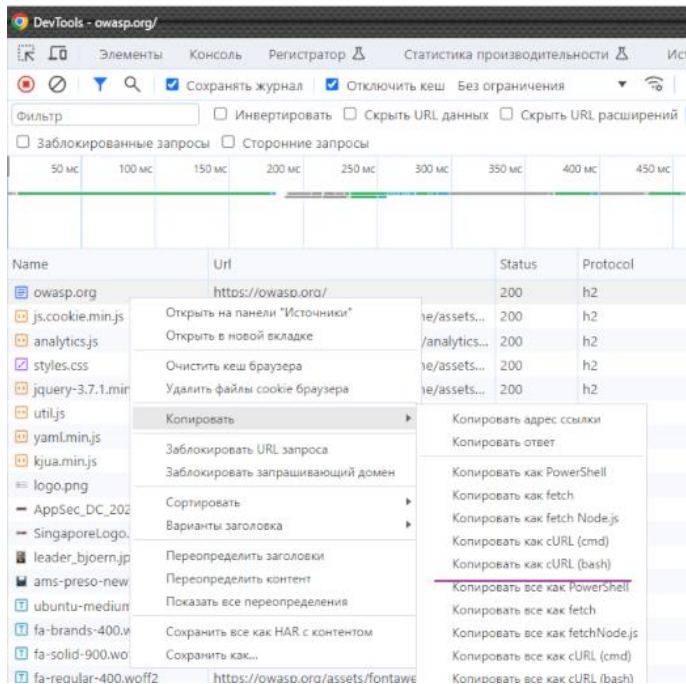
Далее нам нужна вкладка **Сеть**



Отправленные браузером запросы будут отображаться в левой части окна.

Допустим, мы хотим повторить или изменить запросы к сайту owasp.org

Кликаем правой кнопкой по строке owasp.org в колонке Name и выбираем Копировать – Копировать как cURL (bash).



В буфер обмена попадает наш запрос в виде curl строки.

А теперь магия) Открываем сайт <https://curlconverter.com/#> вставляем наш запрос в верхнее поле и получаем полноценный код на python с нужными заголовками

Convert [curl](#) commands to Python, JavaScript and more

curl command

```
-H 'sec-fetch-dest: document' \
-H 'sec-fetch-mode: navigate' \
-H 'sec-fetch-site: none' \
-H 'sec-fetch-user: ?1' \
-H 'upgrade-insecure-requests: 1' \
-H 'user-agent: Mozilla/5.0 (Windows NT 11.0; Win64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/114.0.5740.203 Safari/537.36' \
--compressed
```

Ansible C# ColdFusion Clojure Dart Elixir Go HAR HTTP HTTPie Java JavaScript JSON Kotlin MATLAB Node.js
Objective-C OCaml PHP PowerShell Python R Ruby Rust Swift Wget

```
import requests

headers = {
    'authority': 'owasp.org',
    'accept': 'text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9',
    'accept-language': 'ru-RU,ru;q=0.9,en-US;q=0.8,en;q=0.7,ja;q=0.6,bg;q=0.5',
    'cache-control': 'no-cache',
    'dnt': '1',
    'pragma': 'no-cache',
    'sec-ch-ua': '"(Not A:Brand);v=99", "Chromium";v="114", "Google Chrome";v="114"',
    'sec-ch-ua-mobile': '?0',
    'sec-ch-ua-platform': 'Windows',
    'sec-fetch-dest': 'document',
    'sec-fetch-mode': 'navigate',
    'sec-fetch-site': 'none',
    'sec-fetch-user': '?1',
    'upgrade-insecure-requests': '1',
    'user-agent': 'Mozilla/5.0 (Windows NT 11.0; Win64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/114.0.5740.203 Safari/537.36',
}

response = requests.get('https://owasp.org/', headers=headers)
```

[Copy to clipboard](#)

Этот код можно скопировать в PyCharm или сохранить в файл. В большинстве случаев, вы получите сессию аналогичную браузерной.

Работа с API

API - расшифровывается как **Application Programming Interface**, или интерфейс для программирования приложений. API это набор готовых классов, функций, процедур, структур и констант.

API различных продуктов используются программистами для создания приложений, которые будут взаимодействовать друг с другом. У каждого крупного портала обязательно есть свой API-сервис, предоставляющий информацию в той или иной области.

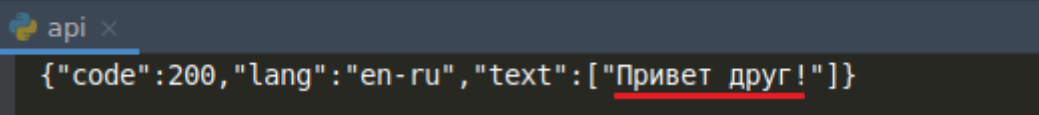
Рассмотрим пример с Yandex Translate API. Зарегистрированные пользователи могут получить ключ на странице <https://yandex.ru/dev/translate/>. Также можно воспользоваться любым другим сервисом, который предоставляет API, чтобы посмотреть как это работает.

У каждого сервиса есть своя документация, в которой можно посмотреть информацию по запросам <https://yandex.ru/dev/translate/doc/dg/reference/translate-docpage/>

С помощью этого сервиса можно переводить тексты. Три параметра передаваемые в dict являются обязательными. Это ключ, текст для перевода, и направление перевода.

```
import requests

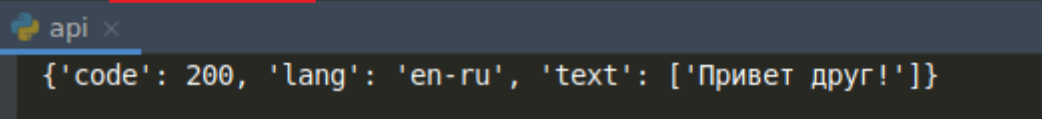
url = 'https://translate.yandex.net/api/v1.5/tr.json/translate'
api = 'trnsl.1.1.20190911T205931Z.32635fcf0918e822.e9aa9ce9787c4'
params = dict(key=api, text='Hello friend!', lang='en-ru')
result = requests.get(url, params=params)
print(result.text)
```



Также мы можем использовать формат json.

```
import requests

url = 'https://translate.yandex.net/api/v1.5/tr.json/translate'
api = 'trnsl.1.1.20190911T205931Z.32635fcf0918e822.e9aa9ce9787c4'
params = dict(key=api, text='Hello friend!', lang='en-ru')
result = requests.get(url, params=params)
print(result.json())
```



Внешне вывод выглядит одинаковым, но при использовании text в выводе будет **class 'str'**, а с json **class 'dict'**. Это более интересный формат, так например, мы легко можем получить нужное нам значение без лишней информации.

```
json = result.json()
print(json['text'][0])
```

api x

Привет друг!

Сервис API предоставляют множество сайтов, и можно смотреть прогноз погоды, курсы валют, и ещё много всего разного. Вот ещё один пример поиска репозитория на GitHub по ключевому слову.

```
import requests

# Поиск местонахождения для запросов на GitHub
response = requests.get(
    'https://api.github.com/search/repositories',
    params={'q': 'shell'},)

# Анализ некоторых атрибутов местонахождения запросов
json_res = response.json()
repository = json_res['items'][0]
print(f'Repository name: {repository["name"]}')
print(f'Repository description: {repository["description"]}')
print(repository["html_url"])

respons x
```

Repository name: **shell**

Repository description: Linux命令行与shell脚本编程大全案例

<https://github.com/fengyuhetao/shell>

Создание запроса POST

Метод `requests.post()` применяется для создания post-запросов.

```
import requests

url = 'https://translate.yandex.net/api/v1.5/tr.json/translate'
api = 'trnsl.1.1.20190911T205931Z.32635fcf0918e822.e9aa9ce9787c4'
params = dict(key=api, text='Try harder!', lang='en-ru')
result = requests.post(url, params=params)
print(result.text)
```

api x

{"code":200,"lang":"en-ru","text":["**Старайся лучше!**"]}

#POST запрос с параметрами через кортеж

```
param_tuples = [('param1', 'value1'), ('param1', 'value2')]
```

```
response = requests.post('https://site.com', data=param_tuples)
```

```
import requests

url = 'https://translate.yandex.net/api/v1.5/tr.json/translate'
api = 'trnsl.1.1.20190911T205931Z.32635fcf0918e822.e9aa9ce9787c4e95'
param_tuples = [('key', api), ('text', 'Dawn'), ('lang', 'en-ru')]
result = requests.post(url, param_tuples)
print(result.text)
```

api x

```
{"code":200,"lang":"en-ru","text":["Рассвет"]}
```

#POST запрос с параметрами через словарь

```
param_dict = {'param1': 'value1', 'param2': 'value2'}
```

```
response = requests.post('https://site.com', data= param_dict)
```

```
import requests

url = 'https://translate.yandex.net/api/v1.5/tr.json/translate'
api = 'trnsl.1.1.20190911T205931Z.32635fcf0918e822.e9aa9ce9787c4e95f49'
data = {'key': api, 'text': 'Morning in the forest', 'lang': 'en-ru'}
result = requests.post(url, data)
print(result.text)
```

api x

```
{"code":200,"lang":"en-ru","text":["Утро в лесу"]}
```

Модуль requests работает со всеми видами запросов:

```
>>> requests.put('https://site.com', data={'key':'value'})
>>> requests.delete('https://site.com')
>>> requests.head('https://site.com')
>>> requests.patch('https://site.com', data={'key':'value'})
>>> requests.options('https://site.com')
```

Для работы с бинарными файлами используется **response.content**. Для примера скачаем логотип сайта. Обратите внимание что запись файла идёт в бинарном режиме **wb**.

```

1  import requests
2
3  url = "https://codeby.net/styles/" \
4      "default/codeby/Cobeyby_net_logo_041.png"
5  response = requests.get(url)
6  response.raise_for_status()
7  filename = 'logo.png'
8  with open(filename, 'wb') as file:
9      file.write(response.content)
10 print('Done!')

```

Done!

Если загружаемый файл имеет большой размер, то рекомендуется использовать метод `iter_content(size)`, который возвращает порции данных размером `size` на каждой итерации. А методу `requests.get()` надо передать именованный аргумент `stream=True`, тогда в память не будет загружаться файл целиком. Параметр `chunk_size` определяет количество байтов, которые должны быть прочитаны в памяти.

```

import requests

img = 'https://sun9-16.userapi.com/c855732/v855732183/ed666/SGfr8gMYkt0.jpg'
res = requests.get(img, stream=True)
res.raise_for_status()
with open('Botan.jpg', 'wb') as f:
    for i in res.iter_content(chunk_size=64000):
        print('Received a Chunk')
        f.write(i)
    print('Done!')

```

Received a Chunk
Received a Chunk
Received a Chunk
Received a Chunk
Done!

Рассмотрим ещё некоторые параметры модуля:

Если требуется отключить проверку SSL-сертификата, параметру `verify` нужно присвоить значение `False`.

```
requests.get('https://site.com', verify=False)
```

При отправке запроса, сервера отвечают с разной скоростью, и возможны долгие ответы и зависания. Чтобы программа стабильно работала, и не зависла на

неопределённое время, необходимо установить время ожидания запроса. Для этого используем параметр `timeout`. Значение параметра устанавливает количество секунд ожидания, и может быть как целым числом, так и числом с плавающей точкой.

```
requests.get('https://site.com', timeout=1)
```

При слишком маленьком значении таймаута сервер не успеет ответить и мы получим исключение, которое необходимо перехватить. Все исключения, которые явно вызывает Requests, наследуются от `requests.exceptions.RequestException`.



```
import requests

try:
    response = requests.get('https://codeby.net', timeout=0.1)
    print(response.status_code)
except requests.exceptions.Timeout:
    print('Ошибка таймаута')
```

req_3 x

Ошибка таймаута

Кроме таймаута соединения, есть и таймаут чтения, то есть время ожидания ответа, как только ваш клиент установил соединение. При необходимости его можно установить в кортеже `timeout=(1, 3)`.

Если запрос устанавливает соединение в течение 1 секунды, и получает данные в течение 3-х секунд после установления соединения, то ответ будет 200. Иначе получим вызов исключения `Timeout`.